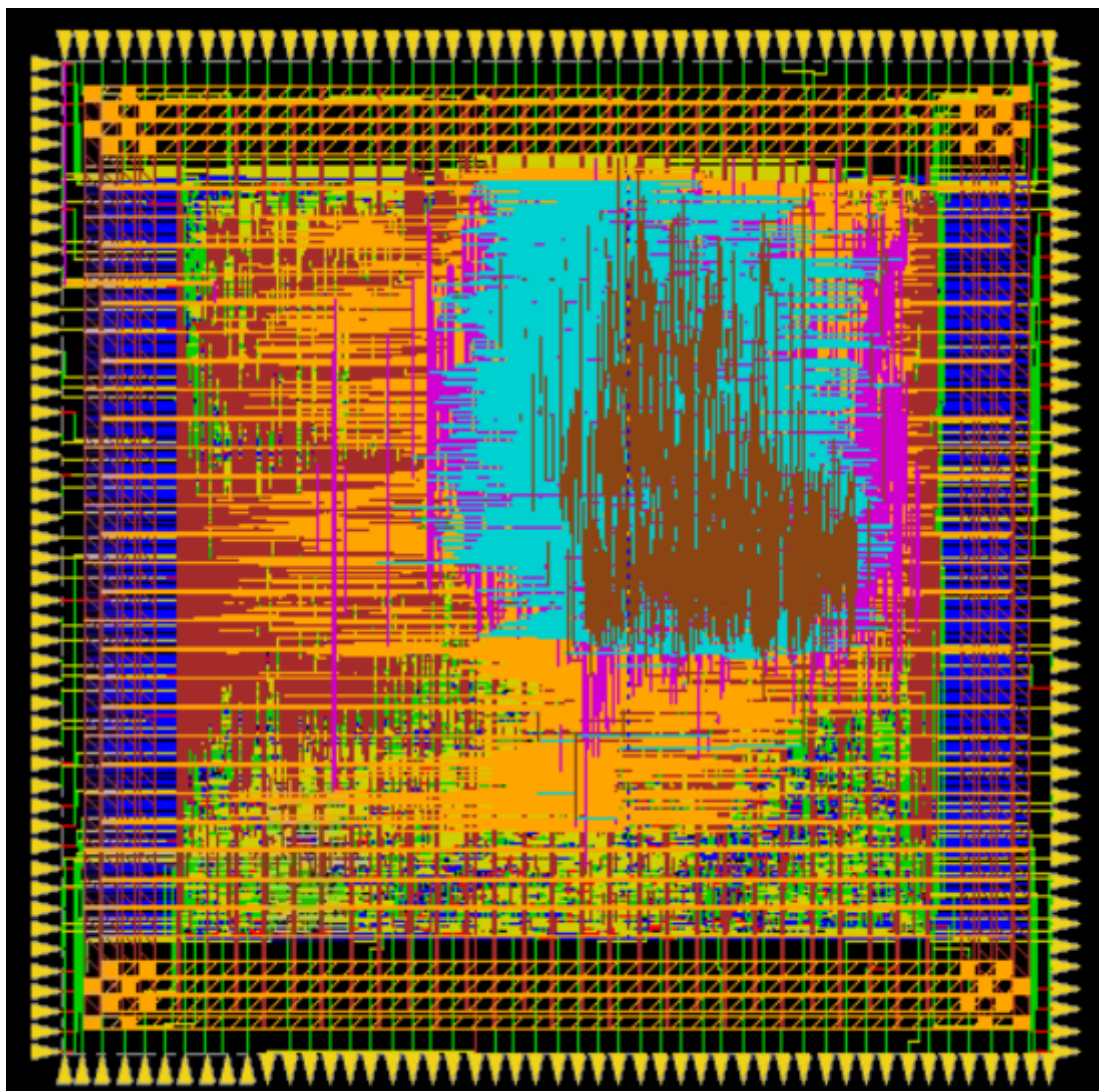


Design Report

B13901080 陳泓瑋 B13901097 鄭允迦

Layout after routing:



1. APR result

Verify DRC result

```
Verification Complete : 0 Viols.  
*** End Verify DRC (CPU: 0:00:04.6  ELAPSED TIME: 5.00  MEM: 256.1M) ***
```

Verify Regular Connectivity result

```
***** End: VERIFY CONNECTIVITY *****  
Verification Complete : 0 Viols.  0 Wrngs.  
(CPU Time: 0:00:00.6  MEM: 0.000M)
```

Analyze Floorplan result

```
***** Analyze Floorplan *****  
Die Area(um^2)      : 18905.15  
Core Area(um^2)     : 11118.44  
Chip Density (Counting Std Cells and MACROs and IOs): 58.812%  
Core Density (Counting Std Cells and MACROs): 100.000%  
Average utilization  : 100.000%  
Number of instance(s) : 37170  
Number of Macro(s)    : 0  
Number of IO Pin(s)   : 194  
Number of Power Domain(s) : 0  
***** Estimation Results *****  
*****
```

2. Configuration

Core area=11118.44, cycle period=0.6ns

3. Testbench result(pass)

Tb1:

```

*****
*                                     *
*          RESULT                     *
*                                     *
*****

# #####
# # * *
# # # CORRECT #
# # #         #
# # #         #
# #####

-----
CONGRATULATION! ALL DATA PASS!
-----

$finish called from file "../00_TESTBED/testbench.v", line 221.
$finish at simulation time          759000
      V C S   S i m u l a t i o n   R e p o r t
Time: 759000 ps

```

simulation time=759000ps

Tb2:

```
*****  
*                                     *  
*                               RESULT                               *  
*                               *****                               *  
  
#   #####  
#   #           #   *   *  
#   #   CORRECT   #   |  
#   #           #   \___/  
#   #####
```

```
CONGRATULATION! ALL DATA PASS!
```

```
$finish called from file "../00_TESTBED/testbench.v", line 221.  
$finish at simulation time          759000  
      V C S    S i m u l a t i o n    R e p o r t  
Time: 759000 ps
```

simulation time=759000ps

Tb3:

```
*****
*                                     *
*                               RESULT                               *
*                               *                                     *
*****

# #####
# # # * *
# # # CORRECT # |
# # # # # \_/_/
# #####

-----
CONGRATULATION! ALL DATA PASS!
-----

$finish called from file "../00_TESTBED/testbench.v", line 221.
$finish at simulation time 670200
V C S   S i m u l a t i o n   R e p o r t
Time: 670200 ps
```

simulation time=670200ps

Tb4:

```
*****
*                                     *
*                               RESULT                               *
*                               *                                     *
*****

# #####
# # # * *
# # # CORRECT # |
# # # # # \_/_/
# #####

-----
CONGRATULATION! ALL DATA PASS!
-----

$finish called from file "../00_TESTBED/testbench.v", line 221.
$finish at simulation time 670200
V C S   S i m u l a t i o n   R e p o r t
Time: 670200 ps
```

simulation time=670200ps

Average simulation time = $(759000 + 759000 + 670200 + 670200) / 4$
= 714600 ps

Performance = $714600 \times 11118.44 = 7.945237224 \times 10^9$ ps·um²

A Design Overview

Main Architecture	Two-line circular buffer + input-row bypass FIFO
Timing Strategy	Lightweight pixel pipeline before the MAC stage
Activation	Clamp, square, then digit-by-digit cube-root computation
Output Addressing	Single raster-order emit_base_addr counter

For the main architecture, the key idea is to use only two full line buffers, combine them with an input-row bypass FIFO, and insert a lightweight pixel pipeline before the MAC (Multiply-ACcumulate convolution) stage. This avoids a large 3-line or 4-line buffer structure while **keeping a throughput of up to four output pixels per cycle**.

A. Workflow / FSM

The workflow is controlled by the following FSM states:

S_IDLE

- Initializes the core and asserts o_in_ready.
- Moves to weight loading.

S_GET_W

- Loads the 3x3 kernel weights into w0~w8.
- Stores stride_reg.
- Clears emit_base_addr, which is later used to generate output addresses.

S_LOAD0 / S_LOAD1

- Loads the first two image rows.
- Row 0 is stored in line 0, row 1 in line 1.
- These two buffers provide the top and middle rows for later convolution windows.

S_PRE_ROW0

- Computes output row 0.
- The top row is outside the image, so it is treated as zero padding.
- Middle and bottom rows are read from the two-line buffers.
- The stride-1 path calls `push_group_from_line_bottom()`, while the stride-2 path calls `push_group_s2_from_line_bottom()`.

S_STREAM

- Main streaming state.
- Each input word contains 4 pixels.
- `prev2_grp`, `prev1_grp`, and `i_in_data` form a small 3-group FIFO for the current input row.
- Top and middle pixels are read directly from the two-line buffers.
- Bottom pixels are generated from the input FIFO/bypass path, so the design does not need to wait until the whole row has been written into a line buffer.
- For stride=1, a 4-output group is produced once enough neighboring input groups are available.
- For stride=2, `read_grp[0]` is used to detect when a pair of input groups is ready, providing the 9 horizontal pixels needed for four stride-2 outputs.

S_ROW_FLUSH

- Handles right-boundary zero padding at the end of each row.
- Writes delayed input groups back into the circular line buffer.
- Rotates `top_sel`, `mid_sel`, and `write_sel`.

S_FINAL_ROW

- Handles the final output row.
- The bottom row is outside the image and is treated as zero padding.

S_DRAIN

- Waits for the internal pipeline to flush.
- This version uses `drain_count == 4'd11`.
(`drain_count` = pipeline flush wait counter)

S_FINISH

- Asserts `o_exe_finish`.

B. Two-Line Buffer + Bypass FIFO

The design keeps only two full line buffers:

```
line0[0:63]
line1[0:63]
```

A 3x3 convolution window needs three rows. Instead of storing three full rows, we use:

- top row from one line buffer,
- middle row from the other line buffer,
- bottom row from the current input stream through a bypass FIFO.

Bypass FIFO consists of:

```
prev2_grp
prev1_grp
i_in_data
```

Each group contains 4 pixels, so three groups provide enough horizontal pixels for both stride=1 and stride=2 windows. This reduces storage and avoids a large sliding-window register structure.

C. Pixel Pipeline and Lane-Pixel Format

A previous row-sum version placed pixel selection, multiplication, and row summation in one cycle:

```
pixel select -> multiplier -> row_sum
```

That path was too long. Hence, we split it into:

```
Cycle A: select pixels -> pixel registers
Cycle B: multiply + row_sum -> sum registers
```

The current implementation also uses a lane-pixel format:

```
pix_t0~pix_t3[0:2]
pix_m0~pix_m3[0:2]
pix_b0~pix_b3[0:2]
```

Each lane already contains its own 3x3 pixel window. Therefore, the MAC stage does not need a stride-dependent mux.

For stride=1:

- lane0: pixels 0,1,2
- lane1: pixels 1,2,3
- lane2: pixels 2,3,4
- lane3: pixels 3,4,5

For stride=2:

- lane0: pixels 0,1,2
- lane1: pixels 2,3,4
- lane2: pixels 4,5,6
- lane3: pixels 6,7,8

The stride difference is handled in the pixel stage, making the MAC stage simpler and more timing friendly.

D. Convolution and Activation

For each output pixel:

- **mul_pixel_weight()** Converts the unsigned pixel to signed form and multiplies it by a signed weight.
- **row_sum3()** Adds three products from one row.
- **row_total3()** Adds the top, middle, and bottom row sums to form the convolution result.

Then the activation is applied:

- **clamp_acc()** Rounds by $(acc + 64) \ggg 7$, then clamps the result to 0~255.
- **square_u8()** Computes x^2 .
- **Digit cube-root pipeline** Computes $x^{2/3} = \text{cube_root}(x^2)$.

The root is constructed bit by bit with six candidate bits: $b = 32, 16, 8, 4, 2$, and 1.

At each stage, the circuit tests whether the candidate root $r + b$ is still valid:

if $(r + b)^3 \leq \text{target}$, accept the bit; otherwise, keep r unchanged.

To avoid using a general multiplier in every stage, the design also keeps r^2 and

r^3 , then updates the candidate values using binomial expansion:

$$(r + b)^2 = r^2 + 2rb + b^2$$

$$(r + b)^3 = r^3 + 3r^2b + 3rb^2 + b^3$$

Since b is a fixed constant in each stage, the terms involving b can be implemented by shifts and adders. For example, when $b = 16$, $2rb$ becomes $r \ll 5$, b^2 is 256, and b^3 is 4096.

After all six stages, r is the largest 6-bit integer whose cube does not exceed target, so the result is exactly $\lfloor \sqrt[3]{target} \rfloor$. Consequently, instead of a LUT, this is an arithmetic digit-by-digit cube-root implementation.

E. Output Address Counter

Instead of passing address registers through every pipeline stage, this version uses one output counter:

```
emit_base_addr
```

When `pipe_valid[5]` is asserted, the core outputs:

```
addr1 = emit_base_addr
addr2 = emit_base_addr + 1
addr3 = emit_base_addr + 2
addr4 = emit_base_addr + 3
```

Then `emit_base_addr` increments by 4. Since the output order is raster order, this removes unnecessary address pipeline registers.

F. Area / Timing Trade-Offs

The main trade-off is adding a small number of pixel registers to reduce the critical path.

The row-sum-only version used fewer registers, but its critical path was:

```
pixel selection + multiplier + row_sum
```

Using a tight 0.6 ns clock, synthesis may need larger cells or duplicated logic to meet timing, increasing area.

The pixel-pipeline version adds pixel registers, but shortens the path:

```
pixel selection -> FF  
multiplier + row_sum -> FF
```

This reduces timing pressure, allowing synthesis to use smaller logic. As a result, total area can decrease even though the RTL has more registers.

The lane-pixel format adds some structured pixel registers but removes the stride mux before the MAC stage. This makes the arithmetic datapath cleaner and easier to optimize.

Overall, this version balances area and timing by keeping high 4-output throughput, avoiding large line/window storage, avoiding full product registers, and using carefully placed pipeline boundaries.